

Mockito

Dobles de prueba en Java: SRS, SDD y diseño paso a paso de pruebas de caja negra y caja blanca

---

Grupo 10

Main Rocabado Jairo Gael Banda Tellez Fernando Velasquez Ricaldez Rodrigo

29 de septiembre de 2026 08:15 – 08:44

Universidad Mayor de San Simón – Auditoría de Sistemas

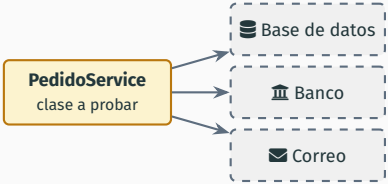
Contenido

1. El problema y los dobles de prueba
2. Mockito: qué es y cómo se usa
3. Arquitectura de Mockito
4. SRS: especificación de requisitos
5. SDD: diseño para ser probado
6. Diseño de casos de prueba paso a paso
7. Caja negra
8. Caja blanca
9. Demo: resultados de la ejecución
10. Conclusiones

Sistema de ejemplo

Un **Servicio de Pedidos** que verifica el stock, calcula descuentos, cobra con un banco y notifica por correo. Lo especificamos (SRS), lo diseñamos (SDD) y lo probamos con Mockito.

El problema y los dobles de prueba



Con dependencias reales, la prueba…

es **lenta**, **frágil** (si el banco cae, falla), tiene **efectos reales** (cobra, envía correos) y no puede provocar un "pago rechazado".

Solución: dobles de prueba

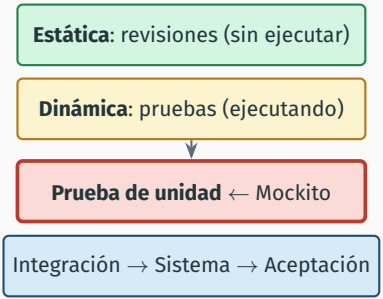
Objetos que **reemplazan** a las dependencias y que el tester **controla**.

Tipo	Qué hace	En Mockito
Stub	responde lo programado	`when()`
Mock	registra llamadas para verificarlas	`verify()`
Spy	objeto real que se espía	`spy(obj)`
Dummy / Fake	relleno / versión simplificada	a mano

Estado: ¿cuál fue el resultado? → caja negra. **Comportamiento**: ¿cómo interactuó? → caja blanca.

No mockeamos la lógica que queremos comprobar, sino las dependencias externas o colaboradoras: inventario, pagos, notificaciones y persistencia.

Mockito en la evaluación de software



Verificación (¿se construyó bien?) → pruebas unitarias.
Validación (¿es lo que se pidió?) → criterios de la SRS.
ISO/IEC 25010: **capacidad para ser probado**.

¿Qué es Mockito?

- Framework **open source** (MIT) para crear mocks, stubs y spies en Java.
- Creado por Szczepan Faber (2008). Es el **estándar de facto**: viene incluido en Spring Boot.
- Filosofía: *primero ejecuta, después verifica*.
- Versión usada: **5.24.0** (Java 11+), con JUnit 6.

Instalación (Maven)

`mockito-core` y `mockito-junit-jupiter`, con `<scope>test</scope>`.

Cómo se usa: crear, programar, ejecutar y verificar

```java
@ExtendWith(MockitoExtension.class)
class PedidoServiceTest {
    @Mock InventarioRepository inventario; // 1. CREAR:
    @Mock PasarelaPago pasarela;           // un mock por
    @Mock Notificador notificador;         // dependencia
    @Mock PedidoRepository repo;
    @Spy  CalculadoraDescuentos calc;      // lógica real
    @InjectMocks PedidoService servicio;   // SUT + mocks

    @Test
    void pedidoAprobado() {
        // 2. PROGRAMAR
        when(inventario.stockDisponible("MON-01")).thenReturn(10);
        when(pasarela.cobrar(anyString(), anyDouble()))
            .thenReturn(ResultadoPago.aprobado("TX-1"));

        // 3. EJECUTAR
        Pedido r = servicio.procesarPedido(pedido, REGULAR);

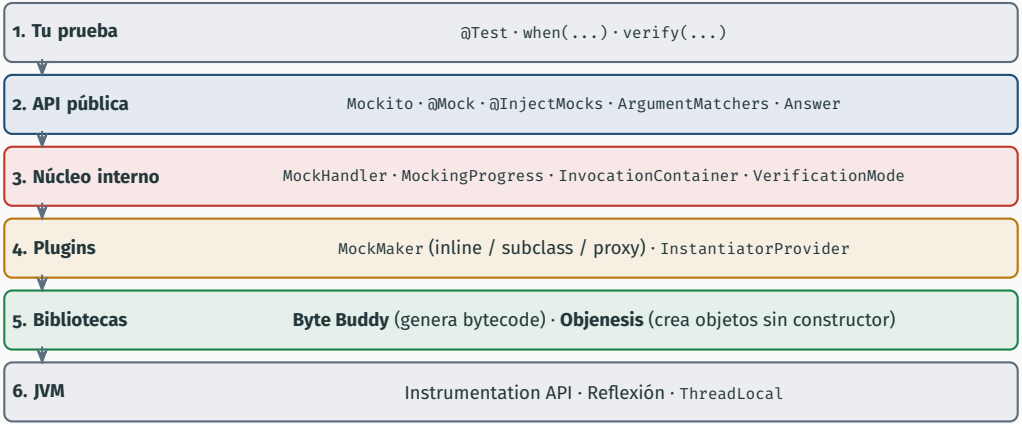
        // 4. VERIFICAR
        assertEquals(PAGADO, r.getEstado());
        verify(inventario).descontarStock("MON-01", 2);
    }
}
```

API	Uso
`when().thenReturn()`	programar respuesta
`thenThrow()`	simular un fallo
`doThrow().when(m)`	métodos `void`
`verify(m, times(n))`	verificar llamadas
`never()`	verificar que no se llamó
`any()`, `eq()`	comodines (matchers)
`ArgumentCaptor`	capturar argumentos
`spy()`, `mockStatic()`	espiar, simular estáticos

Sin programar, un mock devuelve

`0`, `false`, `null`, colecciones vacías u `Optional.empty()`.

Arquitectura por capas

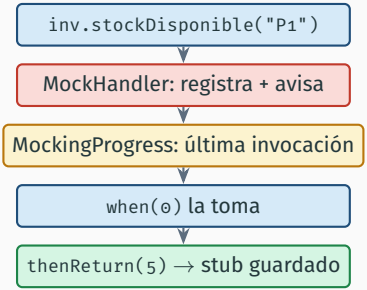


Solo se usa la capa 2 (`org.mockito`). La capa 3 (`org.mockito.internal`) puede cambiar entre versiones. La capa 4 permite **reemplazar piezas** sin tocar Mockito.

Arquitectura: qué pasa en `when(inv.stockDisponible("P1")).thenReturn(5)`

Java evalúa **de dentro hacia fuera**:

1. `inv.stockDisponible("P1")` llega al `MockHandler` (“llamada de ensayo”).
2. Vacía la **pila de matchers**: está vacía ⇒ `"P1"` pasa a `Equals("P1")`.
3. Registra la invocación en el `InvocationContainer`.
4. Avisa a `MockingProgress`: “**esta es la última invocación**”.
5. No hay stub ⇒ devuelve `0` (`RETURNS_DEFAULTS`).
6. `when(0)` ignora el `0` y **toma la última invocación** de `MockingProgress`.
7. `thenReturn(5)` crea un `Returns(5)`, **borra** la llamada de ensayo y **inserta el stub al inicio** de la lista.



Reglas que se derivan

✔ Si un argumento usa un matcher, **todos** deben usarlo: `cobrar(eq("a"), anyDouble())`.
✔ Los métodos `void` se programan con `doThrow().when(mock)`.

SRS – Especificación de requisitos (IEEE 830 / ISO/IEC/IEEE 29148)

¿Qué debe hacer el sistema?

1. Introducción: propósito, alcance, definiciones
2. Descripción general: funciones, actores, restricciones, suposiciones
3. Requisitos específicos:
   - Interfaces externas IE-01…04 (⇒ mocks)
   - Funcionales RF-01…06 con **criterio de aceptación**
   - Reglas de negocio RN-01…04
   - Casos de uso CU-01, CU-02
   - No funcionales RNF-01…05
4. Observaciones de la revisión

→ **Base de prueba**: cada caso de caja negra nace de una frase de la SRS.

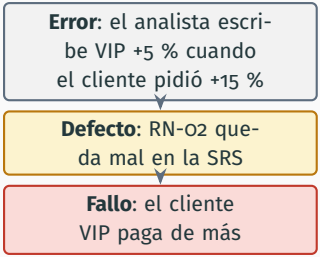
ID	Requisito
RF-01	Rechazar pedidos sin productos
RF-02	Verificar el stock de cada item antes de cobrar
RF-03	Calcular el descuento (RN-01, RN-02)
RF-04	Pago rechazado → RECHAZADO, notificar, no tocar inventario
RF-05	Pago aprobado → descontar, PAGADO, registrar, confirmar
RF-06	Consultar un pedido por id
RN-01	$<$ 100 → 0 %; 100–499,99 → 5 %; $\geq$ 500 → 10 %; negativo inválido
RN-02	VIP: +5 puntos en cualquier tramo
RN-03	Stock suficiente si stock $\geq$ cantidad
RN-04	Total = subtotal − descuento; 2 decimales
RNF-01/02/03	Testabilidad · cobertura de ramas $\geq$ 90 % · sin sistemas reales

SRS – Criterios verificables y revisión estática

RF-05: criterio de aceptación

Dado 2 monitores de 250 Bs, cliente REGULAR, y un banco que aprueba, **cuando** se procesa, **entonces** se cobran **450 Bs**, el estado es PAGADO y se descuentan 2 unidades.

$\Rightarrow$ se convierte directamente en una prueba.



La caja negra toma sus resultados esperados de la SRS: si la SRS falla, la prueba también. Por eso se revisa.

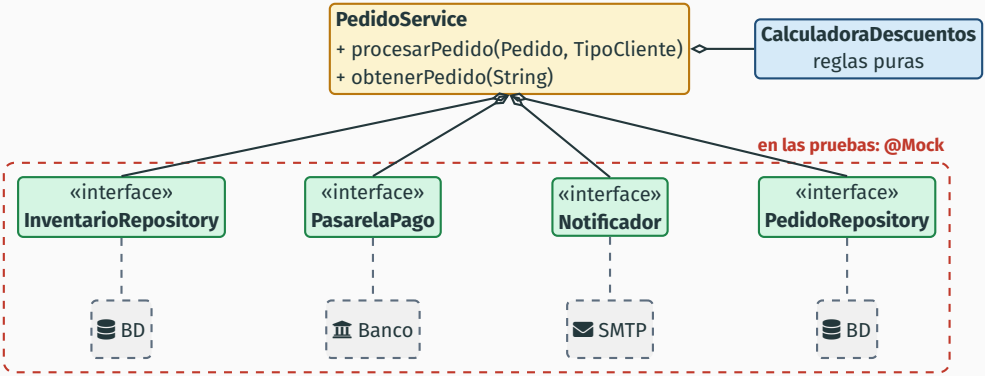
Revisión con lista de comprobación

Criterio	Pregunta	Hallazgo
Completitud	¿Se especificó todo el problema?	OBS-01 pedido nulo
Completitud	¿Se validan todas las entradas?	OBS-02 cantidad $\leq$ 0
Ambigüedad	¿Algún dato admite interpretación?	OBS-03 tipo nulo
Consistencia	¿Qué pasa si falla el registro después de cobrar?	OBS-04
Trazabilidad	¿Cada RF tiene diseño y prueba?	sí (matriz)

🔍 4 defectos encontrados **antes de ejecutar nada**: probar temprano ahorra tiempo y dinero.

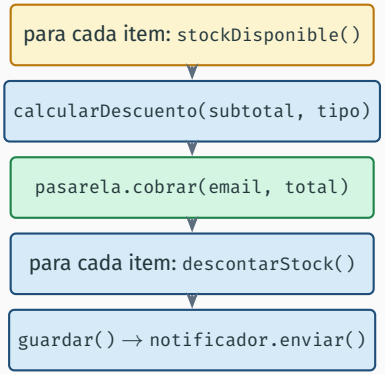
SDD – Diseño para ser probado (IEEE 1016)

Puntos de vista: contexto, lógico, dependencias, interacción, estados y algoritmo. Decisión clave: **todo sistema externo es una interfaz inyectada por constructor**.



Inversión de dependencias + inyección por constructor ⇒ `@InjectMocks` crea el servicio con 5 mocks **sin tocar el código de producción**. Así se cumple la *capacidad para ser probado* de ISO/IEC 25010.

SDD – Interacción, estados y trazabilidad

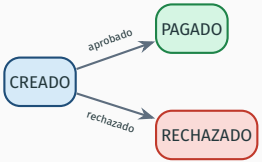


Matriz de trazabilidad

Req.	Casos de prueba
RF-01	R1 | C2
OBS-01	C1 (no estaba en la SRS)
RF-02	R2, L1 | C3, C4
RF-03	CD-01…15 | K1–K5
RF-04	R3 | C5 | E06
RF-05	R4 | C6, C7 | Paso4/5
RF-06	E02
RNF-02	JaCoCo: 100 % de ramas

R, L, CD = caja negra C, K = caja blanca

Estados del pedido



¿Cómo diseñamos cada caso de prueba? (ISO/IEC/IEEE 29119-4)

1. Elemento de prueba y dependencias: ¿qué es mock y qué es real?

2. Base de prueba y técnica: SRS → caja negra; código → caja blanca

3. Condiciones de prueba: rangos, reglas, decisiones

4. Elementos de cobertura: clases, límites, reglas, caminos (numerados)

5. Casos: datos + **resultado esperado calculado a mano** + **qué responde cada mock**

6. Implementar: Arrange (datos + stubs) – Act – Assert (+ `verify`)

7. Ejecutar, analizar el fallo, medir cobertura; volver a 4 si falta algo

Ficha de cada caso (IEEE 829)

ID · origen/técnica · precondiciones · datos de entrada · **dobles (stubs)** · pasos · **resultado esperado** · verificaciones · resultado obtenido

¿Mock o real?

Externo, lento o con efectos (BD, banco, correo) → **mock**.
Lógica propia → **real** o **spy**.
Objetos de valor → `new`.

Cómo escribimos una prueba: de rojo a verde

Objetivo: el criterio de aceptación de RF-05. Cada versión se ejecutó; los resultados son reales.

Versión	Qué agregamos	Resultado real	Qué aprendimos
V1	`@Mock` por dependencia + `@InjectMocks`	✔ verde	Mockito construyó el servicio
V2	datos + llamada + `assertEquals`, sin stubs	✘ “Stock insuficiente”	un mock sin programar devuelve **0**
V3	stub del stock	✘ `NullPointerException`	`cobrar()` devolvió **null**: programar todo el camino
V4	stubs del cobro (450 exacto) y del guardado	✔ verde	el total se calcula a mano *antes*: 500 − 10 % = 450
V5	`verify` del stock, guardado y correo	✔ verde	se verifica también el comportamiento
V6	error del tester: esperar un cobro de 500	✘ “Wanted 500.0 … was 450.0”	la prueba **sí detecta** errores

```
when(pasarela.cobrar("cliente@umss.edu.bo", 450.0))
    .thenReturn(ResultadoPago.aprobado("TX-1"));   // V4
verify(notificador).enviar(eq("cliente@umss.edu.bo"),
        startsWith("Pedido PED-1 confirmado"));     // V5
```

```
Wanted but not invoked:
pasarela.cobrar("cliente@...", 500.0d);
However, there was exactly 1 interaction:
pasarela.cobrar("cliente@...", 450.0d);
```

Caja negra: de la SRS a las clases de equivalencia y los límites

Paso 1. `calcularDescuento` no tiene dependencias ⇒ **sin mocks**.

Paso 2. Base: RF-03, RN-01, RN-02, RN-04.

Paso 3. Entradas: `subtotal` (real, Bs) y `tipo` {REGULAR, VIP}. Salida: descuento con 2 decimales o error.

Paso 4a. Clases (reglas de Myers: rango → una clase por tramo + inválidas; conjunto → una por valor):

CE1	$s < 0$	inválida	RN-01
CE2	$0 \leq s < 100$	0 %	RN-01
CE3	$100 \leq s < 500$	5 %	RN-01
CE4	$s \geq 500$	10 %	RN-01
CE5 / CE6	REGULAR / VIP	+0 / +5	RN-02

Paso 4b. Límites: frontera b y $b - 0{,}01$ (2 decimales, RN-04):

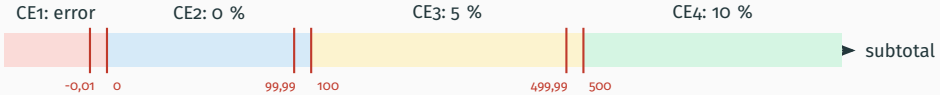
Frontera	$b - 0{,}01$	b
0	−0,01 (CE1)	0 (CE2)
100	99,99 (CE2)	100 (CE3)
500	499,99 (CE3)	500 (CE4)

Paso 4c. Combinación:

- REGULAR con todos los representantes y límites (8)
- VIP: cada tramo al menos una vez (4), porque RN-02 es aditiva
- Cada caso inválido cubre **una sola** clase inválida (2)

Paso 5. Oráculo a mano: p. ej., CD-06:
$499{,}99 \times 0{,}05 = 24{,}9995 \Rightarrow$ **25,00**. También prueba el *redondeo*.

Caja negra: de la tabla de casos al código




```java
@ParameterizedTest(name = "[{index}] {0}: subtotal={1}, {2} -> descuento={3}")
@CsvSource({
    "CE2 límite superior,  99.99,  REGULAR, 0.00",
    "CE3 límite inferior,  100,    REGULAR, 5.00",
    "CE3 límite superior,  499.99, REGULAR, 25.00",
    "CE4 límite inferior,  500,    REGULAR, 50.00",
    "CE4 VIP límite inf.,  500,    VIP,     75.00"      // ... 12 casos válidos
})
void clasesValidasYValoresLimite(String caso, double subtotal, TipoCliente tipo, double esperado) {
    assertEquals(esperado, calculadora.calcularDescuento(subtotal, tipo), 0.001, caso);
}
```

12 válidos + 2 inválidos (`-0.01`, `-100`) + 1 propiedad (VIP > REGULAR) = **15 casos** ✔

Caja negra: tabla de decisión, de 8 reglas a 4

Paso 3. Del CU-01: condiciones C1 ¿items?, C2 ¿stock?, C3 ¿pago aprobado?

Paso 4a. $2^3 = 8$ combinaciones:

	1	2	3	4	5	6	7	8
C1	N	N	N	N	S	S	S	S
C2	N	N	S	S	N	N	S	S
C3	N	S	N	S	N	S	N	S
Acción	A1	A1	A1	A1	A2	A2	A3	A4

Paso 4b. Misma acción + condición indiferente $\Rightarrow$ se fusionan: 1–4 $\rightarrow$ **R1** (sin items, lo demás no importa) · 5–6 $\rightarrow$ **R2** · 7 $\rightarrow$ **R3** · 8 $\rightarrow$ **R4**

Paso 4c. Límite de C2 (RN-03 "$\geq$"): **L1** stock = cantidad.

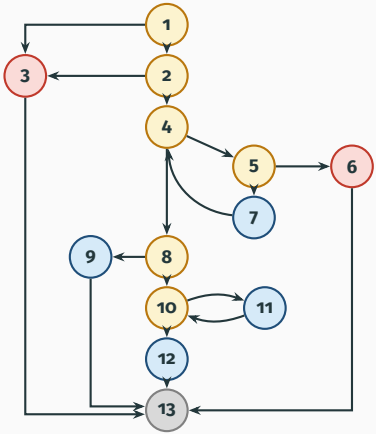
Paso 5: ¿cómo se fuerza cada condición?

C1 **datos**: `List.of()` o 1 item
C2 **stub**: `stockDisponible` $\rightarrow$ 1 o 10
C3 **stub**: `cobrar` $\rightarrow$ rechazado / aprobado

Caso	Stubs	Esperado
R1	ninguno	error sin items
R2	stock 1	error de stock
R3	stock 10, rechazado	RECHAZADO
R4	stock 10, aprobado, guardar	PAGADO, 450
L1	stock 2, aprobado, guardar	PAGADO

Sin `verify`: en caja negra solo se miran las salidas.

Caja blanca: del código al grafo y a $V(G)$



Paso 1. Marcar las decisiones de `procesarPedido`. El `||` se separa en dos:

Nodo	Decisión	Depende de
1 (D1)	`pedido == null`	datos
2 (D2)	`items.isEmpty()`	datos
4 (D3)	`for` validar items	datos
5 (D4)	`stock < cantidad`	**stub** del inventario
8 (D5)	`!pago.aprobado()`	**stub** de la pasarela
10 (D6)	`for` descontar stock	datos

Paso 2. Sentencias seguidas = 1 nodo; un nodo de salida (13).

Paso 3. Complejidad ciclomática

$V(G) = E - N + 2 = 18 - 13 + 2 = \mathbf{7}$ $\quad V(G) = P + 1 = 6 + 1 = \mathbf{7}$ $\quad R = \mathbf{7}$

$\Rightarrow$ **7 caminos independientes = 7 casos de prueba**

● decisión ● excepción

Caja blanca: derivar los caminos y forzarlos con Mockito

Paso 4. Se toma el camino normal como base y se **invierte una decisión a la vez**. **Paso 5.** Se buscan los datos y stubs que obligan a recorrerlo.

Camino	Cómo se obtuvo	Nodos	Datos	Stubs que lo fuerzan
C6	**base**: 1 item, aprobado	1-2-4-5-7-4-8-10-11-10-12-13	P3 × 1	stock 1; cobrar → **aprobado**
C1	invertir D1	1-3-13	`null`	–
C2	invertir D2	1-2-3-13	sin items	–
C3	invertir D4	1-2-4-5-6-13	P1 × 5	stock **4**
C5	invertir D5	1-2-4-5-7-4-8-9-13	P1 × 2	stock 2; cobrar → **rechazado**
C4	2 vueltas, D4 en la 2.ª	1-2-4-5-7-4-5-6-13	P1, P2	stock P1 10, P2 **0**
C7	3 vueltas en cada bucle	1-2-4-(5-7-4)×3-8-…-12-13	3 items	stock 100; aprobado

Con 5 caminos: 18 de 18 aristas

C6, C1, C2, C3 y C5 ya dan 100 % de ramas; C4 y C7 prueban los bucles con varias vueltas.

Camino imposible

“Segundo bucle con 0 vueltas” no puede ocurrir: si D2 dejó pasar el pedido, tiene items.

Caja blanca: cómo se demuestra el camino recorrido (C5)

Paso 6. Evidencia de cada nodo (se comprueba con `verify`):

Nodo	Se comprueba con
3, 6	`assertThrows` + mensaje
5	`verify(inv).stockDisponible(..)`
9	`verify(notif).enviar(.., "Pago rechazado..")`
11	`verify(inv, times(n)).descontarStock`
no 11	`verify(inv, never()).descontarStock`
12	`verify(notif).enviar(.., "confirmado..")`
no 12	`verify(repo, never()).guardar(any())`

La calculadora se **simula** con un valor fijo para aislar `procesarPedido`: se prueba por separado.

```java
@Test @DisplayName("C5: pago rechazado")
void c5PagoRechazado() {
  when(inventario.stockDisponible("P1"))
      .thenReturn(2);                    // D4 = F
  when(calculadora.calcularDescuento(20.0, REGULAR))
      .thenReturn(0.0);
  when(pasarela.cobrar(EMAIL, 20.0))     // D5 = V
      .thenReturn(rechazado("Sin fondos"));

  Pedido r = servicio.procesarPedido(pedido, REGULAR);

  assertEquals(RECHAZADO, r.getEstado());
  verify(notificador)
      .enviar(EMAIL, "Pago rechazado: Sin fondos");
  verify(inventario, never())            // no pasó
      .descontarStock(anyString(), anyInt()); // por 11
  verify(pedidoRepository, never()).guardar(any());
}
```

Demo: ejecución de las pruebas y cobertura

```
$ ./run.sh cobertura
|- Caja blanca - PedidoService (camino básico) ✔
|  |- C1: pedido null ✔
|  |- C2: pedido sin items ✔
|  |- C3: stock insuficiente en el primer item ✔
|  |- C4: falla en el segundo item ✔
|  |- C5: pago rechazado ✔
|  |- C6: pago aprobado con un item ✔
|  '- C7: pago aprobado con tres items ✔
|- Caja negra - PedidoService (tabla de decisión) ✔
|  |- R1 ✔  R2 ✔  R3 ✔  R4 ✔  Valor límite ✔
   ...
Test run finished after 828 ms
[        68 tests found          ]
[         3 tests skipped        ]
[        65 tests successful     ]
[         0 tests failed         ]
```

30 de la API, 20 de caja negra, 12 de caja blanca y 3 del paso a paso. Las 3 omitidas son las versiones en rojo a propósito.

Reporte de cobertura JaCoCo (captura real):

bo.edu.umss.pedidos.servicio

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
PedidoService		100%		100%	0	10	0	35	0	4	0	1
CalculadoraDescuentos		100%		100%	0	6	0	11	0	2	0	1
Total	0 of 200	100%	0 of 20	100%	0	16	0	46	0	6	0	2

```
41.    public Pedido procesarPedido(Pedido pedido, TipoCliente tipoCliente) {
42. ◆      if (pedido == null || pedido.getItems().isEmpty()) {                    // D1, D2
43.            throw new IllegalArgumentException("El pedido no tiene items");
44.        }
45.
46.        double subtotal = 0;
47. ◆      for (ItemPedido item : pedido.getItems()) {                              // D3
48.            String codigo = item.producto().codigo();
49. ◆          if (inventario.stockDisponible(codigo) < item.cantidad()) {          // D4
50.                throw new StockInsuficienteException(codigo);
51.            }
52.            subtotal += item.subtotal();
53.        }
54.
55.        double descuento = calculadora.calcularDescuento(subtotal, tipoCliente);
56.        double total = subtotal - descuento;
57.
58.        ResultadoPago pago = pasarela.cobrar(pedido.getEmailCliente(), total);
59. ◆      if (!pago.aprobado()) {                                                  // D5
60.            pedido.setEstado(EstadoPedido.RECHAZADO);
61.            notificador.enviar(pedido.getEmailCliente(), "Pago rechazado: " + pago.mensaje());
62.            return pedido;
63.        }
64.
65. ◆      for (ItemPedido item : pedido.getItems()) {                              // D6
66.            inventario.descontarStock(item.producto().codigo(), item.cantidad());
```

◆ = todas las ramas de esa decisión probadas ⇒ **100 % de ramas**.

Demo: ¿las pruebas detectan errores?

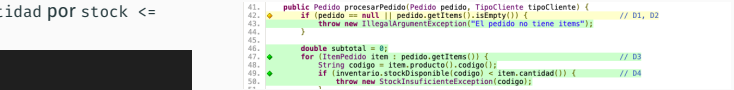
1. Introducimos un error a propósito

En `PedidoService` cambiamos `stock < cantidad` por `stock <= cantidad`:

```
$ ./run.sh
|- Valor límite de stock: stock == cantidad
|  es suficiente ✗ Stock insuficiente para MON-01
|- C5: pago rechazado ✗ Stock insuficiente para P1
|- C6: pago aprobado con un item
|  ✗ Stock insuficiente para el producto P3
|- cobraAntesDeDescontarStock...() ✗ ...MOU-01
[        61 tests successful      ]
[         4 tests failed          ]
```

El caso de **valor límite L1** (caja negra) y los caminos C5 y C6 (caja blanca) detectan el error. Luego restauramos el código y todo vuelve a verde.

2. Cobertura con solo caja negra



♦ **Rombo amarillo** en la línea 42: “1 de 4 ramas sin probar” (`pedido == null`). La SRS no lo menciona (OBS-01); solo la **caja blanca** (camino C1) lo cubre.

Conclusión de la demo

Las dos técnicas se **complementan**: juntas llegan al 100 %.

Conclusiones

1. Mockito aísla la clase bajo prueba: **65 pruebas en menos de 1 s** sin BD, banco ni correo.
2. Su **arquitectura** (`MockHandler`, `MockingProgress`…) explica sus reglas y sus mensajes de error.
3. **SRS** verificable + **SDD** con interfaces inyectadas = requisitos que se pueden probar y lugares donde poner los mocks.
4. Los casos **se diseñan paso a paso** antes de programarse, con el resultado esperado calculado a mano.
5. Caja negra y caja blanca **se complementan**: juntas llegan al **100 % de ramas**.

7 principios que comprobamos

1. Las pruebas muestran defectos, no su ausencia. **2.** Probar todo es imposible: clases, límites y caminos eligen pocos casos. **3.** Probar temprano ahorra: la revisión de la SRS halló 4 defectos antes de programar.

Para la auditoría de sistemas

Pruebas automatizadas + trazabilidad + cobertura = **evidencia objetiva y repetible** (COBIT 2019 BAI03, ISO/IEC/IEEE 29119).

¡Gracias!

Grupo 10 – Mockito